

Overview

To design the ISA, for each of the below program I will have to specify four components:-

1) Instruction Set (Opcodes and Operations):-

- The list of all operations the processor can perform, each identified by an opcode (a numeric code). Operations include arithmetic (ADD, SUB, MUL, DIV), logical (AND, OR, XOR, NOT, SHL, SHR), data movement (LOAD, STORE, MOV), control flow (JMP, BEQ, BNE, CALL, RET), and system operations (interrupt, privilege change).

2) Instruction Formats:- How each instruction is encoded as a bit pattern. The format specifies the width of the instruction (32-bit in MIPS, variable-length in x86) and the fields within it (opcode field, register fields, immediate fields, offset fields). The format design is a core engineering tradeoff: wider instructions allow more expressive operations, but cost code density. Narrower instructions pack more tightly, but limit what you can express in a single operation.

- Firstly as per the assignment I need to use fixed-length instruction format and not variable length (as used in x86). But then in that fixed length of instruction, either I can use fixed-length opcode or variable-length opcode.
- In fixed-length opcode format where all instructions have an opcode field of the same length and at the same position (usually the MSB). For each such instruction I will have to specify two points, opcode and format. For e.g.,
0010 = LOAD
Format: [Opcode][Rd][Address]
This is part of the ISA contract and so our processor decoder just looks at the opcode and knows how to treat the remaining bits or what they represent. Is it address or immediate value, etc.
- If we have a large number of instructions and correspondingly we have multiple decoders then we can have one more abstract layer to categorize instructions such as done in MIPS architecture as R-type, I-type and J-type. For each of these types of instruction we will use a different type of decoder.

To classify our instruction into one of these categories we can either write decoder logic as,

0000 - 0111 = R-type

And then inside a category for each instruction we will have the below

format,

0010 = LOAD

Format: [Opcode][Rd][Address]

Or to classify we can reserve a few bits specifically for instruction type.

For e.g., first two bits will represent,

00 → R-type

01 → I-type

10 → J-type

But this will consume precious instruction bits.

- In variable-length opcode format, we can have following example,
0xxxxxxx → arithmetic instructions
10xxxxxx → memory instructions
110xxxxx → jumps
111xxxxx → special

But here the decoder has to read some bits, decide the type of instruction then read some more bits, which increases the complexity.

3) The Register Set:- Some registers have architectural special roles: the stack pointer (SP), link register (LR, for function return addresses), the zero register (MIPS \$0 / RISC-V x0, always reads as 0). These conventions are part of the ISA specification.

4) Addressing Modes:- How the ISA specifies memory addresses in instructions. Common modes:

- **Register direct:** operand is in a register. ADD R1, R2, R3.
- **Immediate:** constant value embedded in the instruction. ADDI R1, R2, 5 (add 5).
- **Base + offset:** address = register + constant. LW R1, 8(R2) (load from R2+8). The workhorse of RISC memory access.
- **PC-relative:** address = PC + offset. Used for branches and jumps. BEQ R1, R2, +12 (if R1==R2, jump 12 bytes forward).
- **Indirect (CISC):** the register holds the address of the address. MOV AX, [BX+SI*4+8] (x86). Powerful but hardware-expensive to decode.

Programs

Here one important point to note is that it is mentioned in the question that we have a special register PC (6 bits) so we can use instructions with offset value as operand.

Program 1:

Summary of tasks to be done in the program:-

- Load data from memory location 1(3) into R1.
- Load data from memory location 2(5) into R2.
- Add values present in registers(R1 and R2) and save it in the third register(R3).
- Store value present in register(R3) in the memory location 3.

The four components:-

1) Instruction Set:-

- a) Data movement
 - i) LOAD (memory $\rightarrow$ register)
 - ii) STORE (register $\rightarrow$ memory)
- b) Arithmetic
 - i) ADD (register1 + register2 $\rightarrow$ register3)

2) Instruction Formats:-

- a) LOAD: [opcode][Rd][address]
- b) STORE: [opcode][Rs][address]
- c) ADD: [opcode][Rd][Ra][Rb]

3) Addressing Mode:-

- a) Register addressing
- b) Direct addressing

Program 2:

Summary of tasks to be done in the program:-

- Load immediate value 0 in the register named sum.
- Load immediate value 1 in the register named i.
- Subtract 8 - i
- If the carry flag is set, then Branch to instruction where we Store the value in the register named sum in memory location 1.
- Else, we Add sum + i and save it in sum, add i + 1 and save it in i and Jump back to instruction where we subtract 8 - i.

The four components:-

1) Instruction Set:-

a) Data Movement

- i) LOADI (immediate $\rightarrow$ register)
- ii) STORE (register $\rightarrow$ memory)

b) Arithmetic

- i) ADD (register1 + register2 $\rightarrow$ register1)
- ii) ADDI (register1 + immediate $\rightarrow$ register1)
- iii) SUBI (immediate - register)

c) Branch

- i) BC (branch if carry flag is set)
- ii) JMP (jump to previous instruction)

2) Instruction Formats:-

- a) LOADI: [opcode][Rd][immediate]
- b) STORE: [opcode][Rs][memory]
- c) ADD: [opcode][Rd][Ra][Rb]
- d) ADDI: [opcode][Rd][Ra][immediate]
- e) SUBI: [opcode][immediate][Rb]
- f) BC: [opcode][offset]
- g) JMP: [opcode][signed offset]

3) Addressing mode:-

- a) Register addressing
- b) Immediate addressing
- c) Direct addressing
- d) PC-relative

Program 3

Summary of tasks to be done in the program:-

- Load data from memory location 2 to register named sensor_val
- Load immediate value 7 in register named threshold
- Subtract, threshold - sensor_val.
- If the carry flag is set, Branch to the instruction where we Store immediate value 1 in memory location 3.
- Else continue to the next instruction where we Store immediate value 0 in memory location 3.
- Jump one instruction ahead to skip the instruction which is to be executed if the carry flag is set.

The four components:-

1) Instruction Set:-

a) Data Movement

- i) LOAD (memory → register)
- ii) LOADI (immediate → register)
- iii) STOREI (immediate → memory)

b) Arithmetic

- i) SUB (register1 - register2)

c) Branch

- i) BC (branch if carry flag is set)
- ii) JMP (jump to next instruction)

2) Instruction Format:-

- a) LOAD: [opcode][Rd][memory]
- b) LOADI: [opcode][Rd][immediate]
- c) STOREI: [opcode][immediate][memory]
- d) SUB: [opcode][Ra][Rb]
- e) BC: [opcode][offset]
- f) JMP: [opcode][signed offset]

3) Addressing modes:-

- a) Immediate addressing
- b) Direct addressing
- c) Register addressing
- d) PC-relative

Summary

1) Instruction set:-

- a) Data movement
 - i) LOAD (memory $\rightarrow$ register)
 - ii) LOADI (immediate $\rightarrow$ register)
 - iii) STORE (register $\rightarrow$ memory)
 - iv) STOREI (immediate $\rightarrow$ memory)
- b) Arithmetic
 - i) ADD (register1 + register2 $\rightarrow$ register3) OR
ADD (register1 + register2 $\rightarrow$ register1)
 - ii) ADDI (register1 + immediate $\rightarrow$ register1)
 - iii) SUBI (immediate - register)
 - iv) SUB (register1 - register2)
- c) Branch
 - i) BC (branch if carry flag is set)
 - ii) JMP (jump to previous or next instruction)

- Now if we directly use all these instructions then we have a total of **10 instructions** for which we will need 4 bits of opcode. Then we will be left with 4 bits. In this we won't be able to execute instructions requiring addressing three different operands.
- Now in order to solve the problem of avoiding three different operands we can modify those ADD and ADDI instructions so that it takes only two registers, adds the values present in those two registers and then saves the result in the first register. This way these instructions will require only two operands. But even if we do so, then also in one instruction, where we subtract SUBI (immediate - register), there the immediate value is 8 which would need 3 bits but then we will be left with only 1 bit to address the register which is not possible.
- To resolve this problem, we use the following trick: we will load the maximum possible immediate value that we can load with 2 bits, i.e. 4, in one of the registers and then we **introduce a new instruction, "SL", i.e., Shift Left**. This instruction will take a register and shift the value present inside towards left by the amount of shift specified as one of the immediate operands. In this way we will be able to store the value 8 in one of the registers and then perform SUB operation using that register.
- Now we removed one instruction of SUBI and added one instruction of SL, so again we have a total of **10 instructions**. With these modifications our final instruction set along with instruction format will look like:-

1) Instruction Set and Instruction Format along with description:-

a) Data Movement:-

- i) LOAD (memory $\rightarrow$ register)
[opcode] [Rd] [memory]
[4bits] [2bits] [2bits]
Load data present in "memory" location in data memory to register "Rd".
- ii) LOADI (immediate $\rightarrow$ register)
[opcode] [Rd] [immediate]
[4bits] [2bits] [2bits]
Load "immediate" data to register "Rd".
- iii) STORE (register $\rightarrow$ memory)
[opcode] [Rs] [memory]
[4bits] [2bits] [2bits]
Store data present in register "Rd" to "memory" location in data memory.
- iv) STOREI (immediate $\rightarrow$ memory)
[opcode] [immediate] [memory]
[4bits] [2bits] [2bits]
Store the "immediate" value in "memory" location in data memory.

b) Arithmetic

- i) ADD (register1 + register2 $\rightarrow$ register1)
[opcode] [Ra] [Rb]
[4bits] [2bits] [2bits]
Add the values present in register "Ra" and "Rb" and save the result in register "Ra"
- ii) ADDI (register + immediate $\rightarrow$ register)
[opcode] [Rd] [immediate]
[4bits][2bits][2bits]
Add the "immediate" value to the value present in register "Rd" and save the result in register "Rd"
- iii) SUB (register1 - register2)
[opcode] [Ra] [Rb]
[4bits] [2bits] [2bits]
Subtract the value present in register "Rb" from the value present in register "Ra".

c) Branch:-

- i) BC (branch if carry flag is set)
[opcode] [offset]
[4bits] [4bits]
If the carry (C) flag is set, set PC = PC + offset. The carry flag will be set only when we do A - B and A < B.

- ii) JMP (jump to previous / next instruction)
[opcode] [signed offset]
[4bits] [4bits]
Set PC = PC + signed (2's compliment) offset

d) Shift:-

- i) SL (register << immediate)
[opcode] [Rs] [offset]
[4bits] [2bits] [2bits]
Shift the value of register "Rs" left by "offset" times.

- **Nomenclature (opcodes and register id)**

- LOAD: 0000 | LOADi: 0001
- STORE: 0010 | STOREi: 0011
- ADD: 0100 | ADDI : 0101
- SUB: 0110
- BC: 0111
- JMP: 1001
- SL: 1010
- R0: 00 | R1: 01 | R2: 10 | R3: 11

Assembly Codes for the given pseudocodes

- Now lets write the assembly code for all the programs in our assignment

1) Program 1

Program Memory

Address	Instruction	Code
0x00	LOAD R1 MEM[1]	0000 01 01
0x01	LOAD R2 MEM[2]	0000 10 10
0x02	LOAD R3 MEM[0]	0000 11 00
0x03	ADD R1 R2	0100 01 10
0x04	ADD R3 R1	0100 11 01
0x05	STORE R3 MEM[3]	0010 11 11

Data Memory

Address	Data
0x0	0
0x1	3
0x2	5
0x3	-

Static Instruction Count: 6

Dynamic Instruction Count: 6

2) Program 2

Program Memory

Address	Instruction	Code
0x00	LOADI R0 0	0001 00 00
0x01	LOADI R1 1	0001 01 01
0x02	LOADI R2 3	0001 10 11
0x03	SL R2 1	1010 10 01
0x04	ADDI R2 2	0101 10 10
0x05	SUB R2 R1	0110 10 01
0x06	BC 3	0111 0011
0x07	ADD R0 R1	0100 00 01
0x08	ADDI R1 1	0101 01 01
0x09	JMP -5	1001 1011
0x0A	STORE R1 MEM[1]	0010 01 01

Data Memory

Address	Data
0x0	-
0x1	36
0x2	-
0x3	-

Static Instruction Count : 11

Dynamic Instruction Count :

Setup (5) + Loop ((0x05 and 0x06) x 9) + (0x07, 0x08 and 0x09) x 8) + End (1) = **48**

3) Program 3

Program Memory

Address	Instruction	Code
0x00	LOAD R0 MEM[2]	0000 00 10
0x01	LOADI R1 3	0001 01 11
0x02	SL R1 1	1010 01 01
0x03	ADDI R1 1	0101 01 01
0x04	SUB R1 R0	0110 01 00
0x05	BC 2	0111 0010
0x06	STOREI 0 MEM[3]	0011 00 11
0x07	JMP 1	1001 0001
0x08	STOREI 1 MEM[3]	0011 01 11

Data Memory

Address	Data
0x0	-
0x1	-
0x2	-
0x3	-

Static Instruction Count: 9

Dynamic Instruction Count: 7 (if sensor_val > Threshold) or **8** (if sensor_val <= Threshold)

4B. The Immediate Ceiling

The largest constant that my ISA can load into a register using a single instruction is **3**. So in order to load 150 in a register, The assembly code will look like,

```
LOADI R0 3
SL R0 3
SL R0 2
LOADI R1 3
SL R1 3
SL R1 2
ADD R0 R1
LOADI R1 3
SL R1 2
ADD R0 R1
```

So overall I would need **10 instructions**.

4C. Opcode Utilization

Actually I first wrote down what all instructions are needed to execute the tasks in the three programs, then made a decision to modify two instructions (ADD and ADDI) to take two operands in place of three operands, then removed SUBI instruction and instead added SL instruction and finally I made the ISA. So all the instructions I included in my ISA were used in one of the three tasks.

NANO-8 Version 2.0

One change I can make is, I can remove two instructions, **ADDI** and **STOREI** and instead add extra steps of loading the immediate value in one of the registers using **LOADI** instruction and then using **ADD** or **STORE** instructions respectively.

This will help me to reduce the instruction count from 10 to 8 due to which I will have to assign only 3 bits for instruction opcode and remaining bits can be used to increase the range of allowed offset, allowed immediate value or allowed memory address.